PRACTICAL 5. React Routing and Data Handling

PRACTICAL NO.5(A)

Aim:- Install and configure React Router in the React application.

Procedure:-

Step 1: Create a New React Application

Open **Command Prompt (CMD)** and run the following command:

→ npm create vite@latest react-routing-app -- --template react

This command creates a new React application named **react-routing-app** using Vite and the React template.

After executing the command, Vite asks:

Which linter to use?

Select → ESLint

Then it asks:

Install with npm and start now?

Select → Yes

Vite then creates the project and installs the required dependencies. After successful installation, the development server starts automatically.

The CMD displays:

VITE v8.2.2 ready in 241 ms

➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help

This indicates that the React application has been successfully created and is running on the local development server.

Step 2: Open the React Application on Localhost

Open a web browser and visit:

http://localhost:5173/

The default Vite + React application will be displayed.

At this stage, the application is running continuously in the CMD.

Step 3: Stop the Running Development Server

Before installing/configuring additional packages from the same CMD, the currently running Vite development server can be terminated.

Press → Ctrl + C

The CMD may display:

Terminate batch job (Y/N)?

Type → Y

and press **Enter**.

The Vite development server will stop and the normal CMD prompt will appear again.

Step 4: Install React Router DOM

Now install the React Router package using:

npm install react-router-dom

The package will be downloaded and added to the project dependencies.

A successful installation displays information similar to:

added 7 packages in 1s

Step 5: Navigate to the React Project Folder

If the CMD is not already inside the project folder, navigate to the project directory using:

cd react-routing-app

The prompt should now look similar to:

C:\Users\user\react-routing-app>

Step 6: Verify React Router Installation

To check whether react-router-dom has been installed successfully, run:

npm list react-router-dom

The output shows the installed version, for example:

react-routing-app@0.0.0 C:\Users\user\react-routing-app
└── react-router-dom@7.18.3

This confirms that **React Router DOM has been successfully installed** in the React application.

Step 7: Start the React Development Server Again

After installing React Router DOM, start the development server using:

npm run dev

Vite starts the development server and displays:

VITE v8.2.2 ready in 155 ms

➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help

The React application is now running again on:

http://localhost:5173/

Open this address in a browser to verify that the application is working correctly.

Code:-

```
import {BrowserRouter} from "react-router-dom";

function App(){
  return(
    <BrowserRouter>
      <h1>React Router Setup</h1>
    </BrowserRouter>
  );
}
export default App;
```

Output:-



PRACTICAL NO.5(B)

Aim:- Create multiple pages and implement navigation between them.

Code:-

Home.jsx:-

```
function Home() {
   return<h1>Home Page</h1>;
}

export default Home;
```

About.jsx:-

```
function About(){
   return <h1>About Page</h1>;
}

export default About;
```

Contact.jsx:-

```
function Contact(){
   return<h1>Contact</h1>;
}

export default Contact;
```

App.jsx:-

```
import {BrowserRouter,Routes,Route,Link} from "react-router-dom";

import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

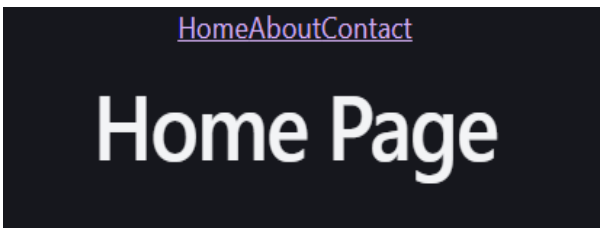
function App() {
  return(
    <BrowserRouter>
      <nav>
        <Link to="/Home">Home</Link>
        <Link to="/About">About</Link>
        <Link to="/Contact">Contact</Link>
      </nav>

      <Routes>
        <Route path="/Home" element={<Home/>}/>
        <Route path="/About" element={<About/>}/>
        <Route path="/Contact" element={<Contact/>}/>
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Output:-









PRACTICAL NO.5(C)

Aim:- Use Link and Routes to move between pages.

Code:-

```
import {BrowserRouter,Routes,Route,Link} from "react-router-dom";

function Home(){
  return <h2>Home Page</h2>
}

function About() {
  return <h2>About Page</h2>
}

function Services() {
  return <h2>Services Page</h2>
}

function App() {
  return(
    <BrowserRouter>
      <h1>React Navigation</h1>

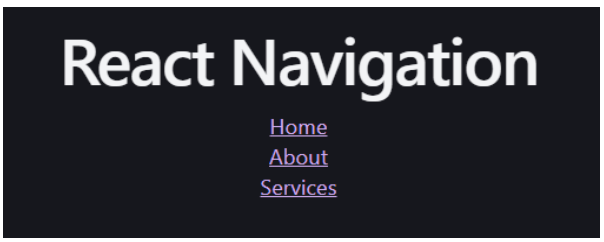
      <Link to="/Home">Home</Link>
      <Link to="/About">About</Link>
      <Link to="/Services">Services</Link>

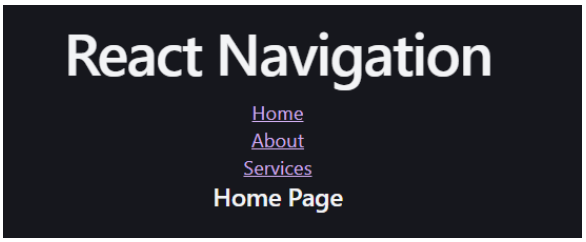
      <Routes>
      <Route path="/Home" element={<Home/>}/>
      <Route path="/About" element={<About/>}/>
      <Route path="/Services" element={<Services/>}/>
      </Routes>

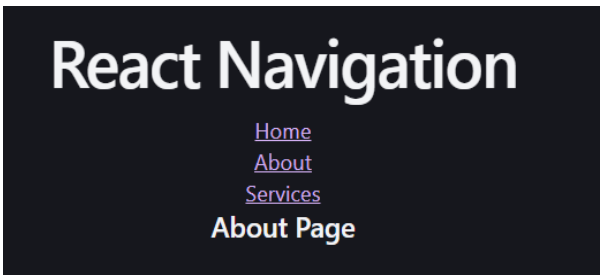
    </BrowserRouter>
  );
}

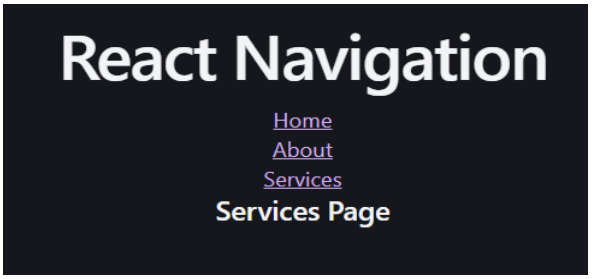
export default App;
```

Output:-









PRACTICAL NO.5(D)

Aim:- Fetch sample data from a public API using Axios.

Code:-

(Install npm axios from new cmd(steps & ss))

```
import {useEffect, useState} from 'react';
import axios from 'axios';

function App() {
  const [user, setUser] = useState([]);

  useEffect(() => {

    axios.get('https://jsonplaceholder.typicode.com/users')

      .then(response => {
        setUser(response.data);
      })

      .catch(error => {
```

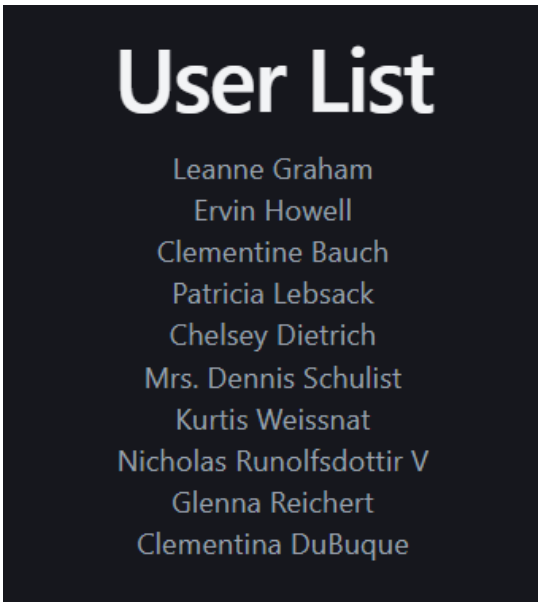
```
        console.error('Error fetching user data:', error);
      });
  }, []);

  return (
    <div>
      <h1>User List</h1>

      {user.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default App;
```

Output:-



PRACTICAL NO.5(E)

Aim:- Display fetched data in table or list format.

Code:-

```
import {useEffect, useState} from 'react';
import axios from 'axios';

function App() {
  const [user, setUser] = useState([]);

  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/users')
      .then(response => {
        setUser(response.data);
      })
      .catch(error => {
        console.error('Error fetching user data:', error);
      });
  }, []);

  return (
    <div>
      <h1>User List</h1>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>
        <tbody>
          {user.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
```

```
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Output:-



User List

ID	Name	Email	City
1	Leanne Graham	Sincere@april.biz	Gwenborough
2	Ervin Howell	Shanna@melissa.tv	Wisokyburgh
3	Clementine Bauch	Nathan@yesenia.net	McKenziehaven
4	Patricia Lebsack	Julianne.OConner@kory.org	South Elvis
5	Chelsey Dietrich	Lucio_Hettinger@annie.ca	Roscoeview
6	Mrs. Dennis Schulist	Karley_Dach@jasper.info	South Christy
7	Kurtis Weissnat	Telly.Hoeger@billy.biz	Howemouth
8	Nicholas Runolfsdottir V	Sherwood@rosamond.me	Aliyaview
9	Glenna Reichert	Chaim_McDermott@dana.io	Bartholomebury
10	Clementina DuBuque	Rey.Padberg@karina.biz	Lebsackbury

PRACTICAL NO.5(F)

Aim:- Implement search or filter functionality on displayed data.

Code:-

```
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [users, setUsers] = useState([]);
  const [search, setSearch] = useState("");

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.error("Error fetching users:", error);
      });
  }, []);

  const filteredUsers = users.filter((user) =>
    user.name.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div>
      <h1>User Search</h1>

      <input
        type="text"
        placeholder="Search by name"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />

      <br />
      <br />

      <table border="1" cellPadding="10">
```

```
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>

        <tbody>
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Output:

